Page No.
Date

Introduction
Instructions
~~8086 ALP~~
8 bit sum of 8 bit numbers
16 bit sum of 8 bit numbers
Flags & Jump statements
16 bit sum of 16 bit numbers
8 bit difference of 8 bit numbers
16 bit difference of 16 bit numbers
To find max or min among a list of N 8 bit numbers
To find max and min among a list of N 8 bit numbers
16 bit product of two 8 bit numbers :
- → by successive addition
- → by shifting multiplicand
- → by shifting product

Division of 16 bit number by 8 bit number
- → by repeated subtraction
- → by shifting dividend

To count positive & negative numbers from a list of N 8 bit numbers
To count odd & even numbers from a list of N 8 bit numbers

Page No.
Date

MicroProcessor

A microprocessor is a multipurpose, programmable, clock-driven, register-based electronic device that reads binary instructions from a storage device called memory, accepts binary data as input and processes data according to those instructions and provide result as output

An instruction is a command given to the computer to perform a specified operation on given data

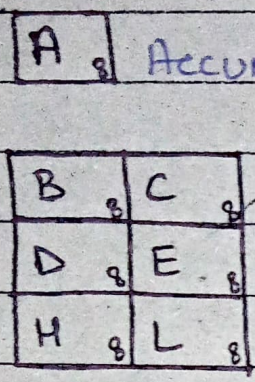
- MVI r/M, XXH (Move immediate data)
  eg: MVI A 00H ; MVI B, 12H ;
  MVI M 02H ;

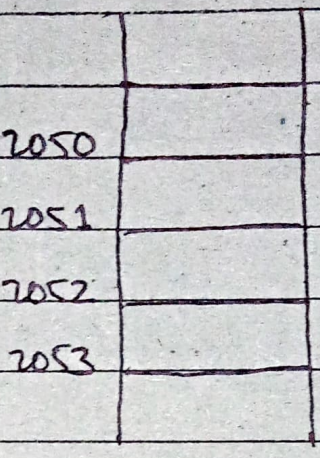
- LXI rp, XXXXH (Load reg. pair with immediate data)
  eg LXI B 1234H ; LXI H 2050H ;

- MOV r/M , r/M (to move (copy) data from source to dest.)
  MOV B, A // Copy content of A to B
  MOV A, M // Load content to A, from mem pointed by HL
  MOV M, A // store content of A, to mem pointed by HL

M refers to the value pointed by HL register pair.

Page No.
Date





- INR r // to increment register value
- INX rp // to increment register pair value

M refers to the value pointed by HL register pair

eg: A 05 | H 20 | L 50

2050 | 04

ADD H ⟶ A = 25

ADD M ⟶ A = 09

- ADD r/M add value (content) of register to accumulator.
- eg: ADD B means A ← A+B

- SUB r/M sub content of register from accumulator
  eg: SUB B means A ← A - B

- LDA address (Load Accumulator) to read from memory
  eg: LDA 2050H

- STA address (store Accumulator) to write on memory
  eg: STA 2052H

Page No.
Date

Q) to add values from 2050H & 2051H and store the sum at 2052H

LDA 2050H	A has first number from 2050H
MOV B, A	first number moved to B
LDA 2051H	A has second number from 2051H
ADD B	A has A+B, i.e., the sum of two numbers
STA 2052H	sum from A stored at 2052H
HLT	stop

Q) to add three 8-bit numbers stored at consecutive memory locations starting from 2050H. And store the sum at 2053H

LXI H, 2050H	
MOV A, M	
INX H	
ADD M	
INX H	
ADD M	
INX H	
MOV M, A	
HLT	

LDA address ⇔ LXI H address; MOV A, M

STA address ⇔ LXI H address; MOV M, A

- LDAX rp // Load the value pointed by reg. pair to the accumulator
  eg: LDAX B

- STAX rp // store the value from accumulator to the memory pointed by the reg. pair.
  eg: STAX D

LXI B, 2050H ; LDAX B ⟺ LDA 2050H

LXI B, 2050H ; STAX B ⟺ STA 2050H

Flags: ALU →

S	Z	x	AC	x	P	x	Cs

Sign Flag ←

Zero Flag ←

Auxillary Carry Flag ←

Pairity Flag ←

Main Carry Flag ←

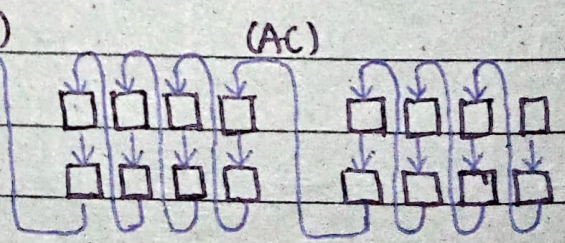
S: 0 for positive, 1 for negative

Z: 0, if result is zero

P: 0 for odd, 1 for even

AC: Carry after 4th bit (for BCD)

Cs: Carry after 8th bit



In Subtraction, Carry flag acts as Borrow flag.

Page No.
Date

Jump Statements : They transfer the program control to a certain memory location conditionally or unconditionally.

	JMP label	Jump unconditionally	
S	JP label	Jump on Plus	(S=0)
S	JM label	Jump on Minus	(S=1)
Z	JZ label	Jump when Zero	(Z=0)
Z	JNZ label	Jump when No Zero	(Z=1)
P	JPO label	Jump on Parity Odd	(P=0)
P	JPE label	Jump on Parity Even	(P=1)
C	JNC label	Jump on No Carry	(Cs=0)
C	JC label	Jump on Carry	(Cs=1)

Q) Write a program to find 8 bit sum of N bit numbers stored at consecutive memory locations.

	LXI H, 2050H	2000 : 21 , 2001 : 50 , 2002 : 20
	MOV C, M	2003 : 4E
	INX H	2004 : 23
	MVI A, 00H	2005 : 3E , 2006 : 00
back :	ADD M	2007 : 86
	INX H	2008 : 23
	DCR C	2009 : 0D
	JNZ back	200A : C2 , 200B : 07 , 200C : 20
	MOV M, A	200D : 77
	HLT	200E : 76/EF

Page No.
Date

Q) Write a program to find 16-bit sum of N 8-bit numbers .....

LXI H, 2050H	HL pointing first location, i.e., no. of numbers
MOV C, M	number count stored in C
INX H	HL now points first number
MVI A, 00H	Lower sum (A) = 0, Upper sum (B) = 0
MVI B, 00H	add current number to A
back: ADD M	if carry is generated, increment B
JNC Next	if carry is not found, don't increment B
INR B	HL now points to next number
next: INX H	decrement the counter by 1
DCR C	if counter is not yet zero, jump to back
JNZ back	if counter is zero, store lower sum to the
MOV M, A	location after the last number.
INX H	HL now points to next location
MOV M, B	store upper sum to the location next to the
RST	lower sum. STOP.

Binary Coded Decimal (BCD)

Binary coded decimal is a system of writing numerals that assign a four-digit binary code to each digit 0 to 9 in a decimal number.

eg: 49 = 0100 1001
(Dec) (BCD)

n-digits in binary becomes 4n-digits in BCD

Dec.	Binary		BCD
00	00000		0000
01	00001		0001
02	00010		0010
03	00011	=	0011
⋮	⋮		⋮
07	00111		0111
08	01000		1000
09	01001		1001
10	01010		0001 0000
11	01011		0001 0001
12	01100		0001 0010
⋮	⋮	+ 0110 =	⋮
17	10001		0001 0111
18	10010		0001 1000
19	10011		0001 1001
20	10100		0010 0000
21	10101		0010 0001

We can use this to implement decimal sum in BCD.

- DAA // convert the sum stored Accum. into a BCD number.

eg: ADD B
DAA

A = 0010 1001 B = 0011 0110
A = A+B = 0101 1111 → 0110 0101

If LD in hex <= 9, AC flag is 0. If 9 < LD in hex <= 19 AC flag is 1, and it adds 0110 to the LD and carry to HD if any.

Page No.
Date

- ADC reg/M // add with carry

eg: ADC B

ADD B	A ← A+B
ADC B	A ← A+B+Carry

Q Write a program to find 16 bit sum of two 16-bit numbers.

```
LDA  2050H
MOV  B,A
LDA  2052H
ADD  B
STA  2054H
LDA  2051H
MOV  B,A
LDA  2053H
ADC  B
STA  2055H
HLT
```

imp: These cmds don't affect Carry flag

2050	L byte	} n_1
2051	U byte	
2052	L byte	} n_2
2053	U byte	
2054	L byte	} sum = n_1+n_2
2055	U byte	

- ADI XXH // add immediate

eg: ADI 05H A ← A+05

- ACI XXH // add immediate with carry.

eg: ACI 07H A ← A+07+Cs

• SUB x/M // subtract	ADD equiv
eg: SUB B A ← A-B	ADD x/M
• SUI XXH // subtract immediate	ADI XXH
eg: SUI 05H A ← A-05	
• SBB x/M // subtract with borrow	ADC x/M
eg: SBB B A ← A-B-Cs	
• SBI XXH // subtract imm. with borrow	ACI XXH
eg: SBI 05H A ← A-05-Cs	

Q Write a program to calc. 16-bit diff. of two 16 bit numbers.

```
LDA 2052H
MOV B,A
LDA 2050H
SUB B
STA 2054H
LDA 2053H
MOV B,A
LDA 2051H
SBB B
STA 2055H
HLT
```

2050	L byte	} n_1
2051	U byte	
2052	L byte	} n_2
2053	U byte	
2054	L byte	} diff $= n_1 - n_2$
2055	U byte	

Page No.
Date

- CMP r/M // Compare the number in the reg, with num in Accu

eg: CMP B C flag = 0 if A >= B
C flag = 1 if A < B

Q Write a program to find the max/min number among N 8-bit numbers.

```
    LXI H,2050H
    MOV C,M
    DCR C
    INX H
    MOV A,M
    INX H
back: CMP M
    JNC next (JC next, for min)
    MOV A,M
next: INX H
    DCR C
    JNZ back
    MOV M,A
    HLT
```

N	number count
1st num	
2nd num	
:	
nth num	

- LHLD XXXX H // Load HL pair direct

eg: LHLD 2050H; L ← 04 H ← 05

- SHLD XXXX H // store HL pair direct

eg: SHLD 2053H; 2053 ← L 2054 ← H

04	2050
05	2051
	2052
04	2053
05	2054

Page No.
Date

Q Write a program to find both min & max numbers among N given 8-bit numbers.

```
    LXI  H, 2050H
    MOV  C, M
    DCR  C
    INX  H
    MOV  B, M
    MOV  D, M
back: INX  H
    MOV  A, B
    CMP  M
    JNC  next1
    MOV  B, M
next1: MOV  A, D
    CMP  M
    JC   next2
    MOV  D, M
next2: DCR  C
    JNZ  back
    INX  H
    MOV  M, B
    INX  H
    MOV  M, D
    HLT
```

Page No.
Date

Q To find 16-bit product of two 8-bit numbers by successive addition

```
      LXI  H,2050H
      MOV  C,M
      INX  H
      MOV  D,M
      MVI  A,00H
      MVI  B,00H
back: ADD  D
      JNC  next
      INR  B
next: DCR  C
      JNZ  back
      INX  H
      MOV  M,A
      INX  H
      MOV  M,B
      HLT
```

2050	mult 1	→ n_1
2051	mult 2	→ n_2
2052	L-byte	} product.
2053	U-byte	} = $n_1 * n_2$

- DAD rp // Double Add, to do 16-bit additions

eg: DAD D ; HL ← HL + DE

- XCHG // exchange content of HL with content of DE

HL ← DE

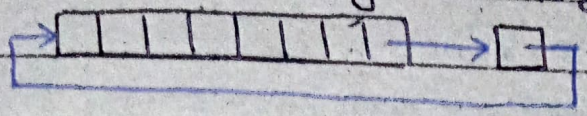
DE ← HL

B5×1
B0×16

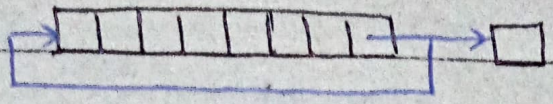
Page No.
Date

rotate commands

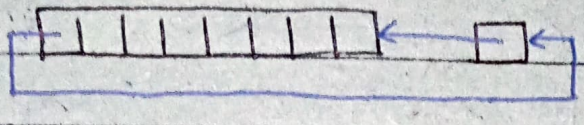
(.8 bits of ~~Accu~~ Accu.) (C flag)

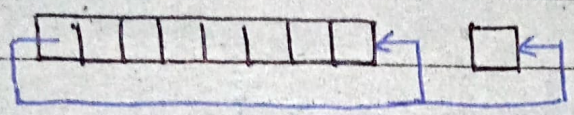
RAR 

can be used to have MSB or LSB of Accum.

RRC 

can be used to rotate the number by 1-bit.

RAL 

RLC 

can be used to double or half a number.

multiplication of two numbers by bit-shifting

eg: 1010 × 1101 can be done in two ways

// Shifting of multiplicand

Let $M_A = 1010$

$M_B = 1101$

~~Intermediate~~ Product

$M_A * M_B(0) * 2^0 =$ 1010

$M_A * M_B(1) * 2^1 =$ 0000x

$M_A * M_B(2) * 2^2 =$ 1010xx

$M_A * M_B(3) * 2^3 =$ 1010xxx

Product 10000010

```
for(i=0,P=0; i<4; i++)
{ P= P+ MA * MB(i) * 2^i ;}
```

// Shifting of product

(Pinitial = 0000) + $M_A * M_B(3)$: 00000000 + 1010 = 00001010

~~Left shift product~~ + $M_A * M_B(2)$: 00010100 + 1010 = 00011110

Left shift product + $M_A * M_B(1)$: 00111100 + 0000 = 00111100

Left shift product + $M_A * M_B(0)$: 01111000 + 1010 = 10000010

Product = 10000010 $(130)_{10}$

Page No.
Date

Q To find 16-bit product of two 8 bit numbers (shifting multiplicand)

LHLD 2060H	H ← multiplier L ← multiplicand
MOV E,L	D ← 00 E ← multiplicand
MVI D,00H	
MOV A,H	A ← multiplier
MVI C,08H	C ← 08 (counter value)
LXI H,0000H	HL ← 0000 (product)
back: RAR	rotating multiplier right to get its bits in C flag one by one (LSB first)
JNC next	if $M_B(i)==1$, add multiplicand to product
DAD D	else do nothing;
next: XCHG	These 3 cmds collectively work to shift multiplicand left by 1 bit (by doubling it).
DAD H	
XCHG	
DCR C	One iteration is over, just repeat this
JNZ back	block 8 times, since multiplier has 8 bits.
SHLD 2062H	Store the 16-bit product from HL pair to memory
HLT	

X

Page No.
Date

d)

Q To find 16-bit product of two 8-bit numbers (by shifting the product)

```
        LHLD  2050H
        MOV   E,L
        MVI   D,00H
        MOV   A,H
        MVI   C,08H
        LXI   H,0000H
back:   DAD   H
        RAL
        JNC   next
        DAD   D
next:   DCR   C
        JNZ   back
        SHLD  2052H
        HLT
```

D ← 00H E ← multiplicand
C ← 08H A ← multiplier
HL ← 0000H (product)

Shift product left by 1 bit (doubling)
rotating multiplier left to get its bits in
C flag one by one (MSB first)
if $m_B(j) == 1$, add multiplicand to product
one iteration is over, total 8 iterations
repeat the loop while (counter > 0)

store the 16 bit product from HL pair
to the memory addresse

Page No.
Date

Q To Divide an 8-bit number by another 8-bit number.
(by repeated subtraction)

```
        LXI  2050H
        MOV  A,M
        INX  H
        MOV  B,M
        MVI  C,00H

back:   CMP  B
        JC   out
        SUB  B
        INR  C
        JMP  back

out:    INX  H
        MOV  M,C
        INX  H
        MOV  M,A
        HLT
```

```
                                A ← Dividend  (later remainder)
                                B ← divisor
                                C ← 00H       (later quotient)

while
if (B<A)        (divisor < Dividend)
{
    A = A-B;    (Dividend -= divisor)
    C++;        (Quotient += 1)
}
then
{
    store C;    (store Quotient)

    store A;    (store Remainder)
}
```

Page No.
Date

Q, To divide a 16-bit* number by an 8-bit# number (by shifting dividend)

```
        LHLD    2050H       HL ← 16 bit Dividend
        LDA     2052H
        MOV     B,A         B ← 8 bit Divisor
        MVI     C,08H
back:   DAD     H
        MOV     A,H
        SUB     B
        JC      next
        MOV     H,A
        INR     L
next:   DCR     C
        JNZ     back
        SHLD    2053H       H ← remainder   L ← Quotient
```

2050	Divd (LB)
2051	Divd (UB)
2052	divisor
2053	Quotient ←
2054	remainder ←

* we can have only 15 bit for the Divident, the MSB must be 0

MSB of the divisor must be 1, for this algorithm to work well.

* here positive means non-negative, i.e., zero is considered as positive

Page No.
Date

Q To count positive* & negative from list of N 8 bit numbers

```
      LXI  H,2050H
      MOV  C,M        C ← number Count
      MVI  B,00H      B ← positive Count
      MVI  D,00H      D ← negative Count
back: INX  H
      MOV  A,M
      JM   next1
      INR  B
next1: JP  next2
      INR  D
next2: DCR  C
      JNZ  back
      INX  H
      MOV  M,B
      INX  H
      MOV  M,D
      HLT
```

you can also achieve this by using RAL to get MSB of the number in accumulator
if (MSB == 0) → number is positive
if (MSB == 1) → number is negative

Page No.
Date

• Q To count odd & even numbers from a list of N 8-bit numbers.

```
        LXI  H, 2050H
        MOV  C, M          C ← number Count
        MVI  B, 00H        B ← even Count
        MVI  D, 00H        D ← odd Count
back:   INX  H
        MOV  A, M
        RAR
        JC   next1
        INR  B
next1:  JNC  next2
        INR  D
next2:  DCR  C
        JNZ  back
        INX  H
        MOV  M, B
        [illegible]
        INX  H
        MOV  M, D
        HLT
```